



Laboratorio No. 1 Semana #13 – APIS con swagger & postman

Nombre: __Pablo David Molina Son____ **Carné:** __1338825____

Objetivo: Comprender y consumir una API REST mediante Swagger UI y Postman, identificando endpoints, métodos HTTP, parámetros, cuerpos JSON, códigos de respuesta y operaciones CRUD. El estudiante deberá interpretar la documentación de una API, ejecutar solicitudes y resolver un reto práctico utilizando evidencia técnica.

Investigación previa:

1. ¿Qué es una API REST?

Una API REST es una forma estándar de diseñar servicios web para que diferentes programas hablen entre sí usando HTTP. Su regla principal es que no guarda estado: cada petición que envías debe llevar toda la información necesaria para procesarse, sin depender de que el servidor recuerde lo que hiciste antes.

2. ¿Qué es un endpoint?

Un endpoint es la ruta o punto de acceso específico dentro del servidor al que apuntas para interactuar con un recurso. Le indica a la API a qué función o dato exacto quieres llegar.

3. ¿Qué diferencia existe entre una URL y un endpoint?

URL: Es la dirección web completa que te lleva al servidor.

Endpoint: Es el destino lógico dentro de esa dirección junto con la acción que vas a realizar.

Una misma dirección base puede tener varios endpoints según el verbo HTTP o el camino final al que apuntes.

4. ¿Qué función cumplen los métodos HTTP GET, POST, PUT/PATCH y DELETE?

GET: Sirve para consultar y leer información sin modificar nada en la base de datos.

POST: Envía datos para crear un nuevo registro o recurso en el servidor.

PUT: Reemplaza por completo un registro existente con los datos nuevos que mandas.

PATCH: Modifica solo uno o varios campos específicos de un registro sin tocar el resto.

DELETE: Borra el recurso indicado en el servidor.

5. ¿Qué es JSON y por qué es común en las APIs?

JSON es un formato de texto plano basado en pares de clave y valor para enviar datos estructurados. Es el estándar porque es sumamente ligero, cualquier lenguaje de programación lo procesa de forma nativa sin complicaciones y resulta muy fácil de leer y depurar cuando estás programando.

6. ¿Qué es un código de estado HTTP? Mencione ejemplos de 200, 404 y 500.

Es una respuesta numérica de tres dígitos que el servidor te devuelve para decirte qué pasó con tu petición. 200 OK: La petición se resolvió con éxito y el servidor te manda la respuesta esperada. 404 Not Found: La ruta no existe o el registro que buscabas no se encontró en el servidor. 500 Internal Server Error: Ocurrió una falla inesperada o tronó el código del servidor al procesar la solicitud.



7. ¿Qué información se puede encontrar en la respuesta de una API?

Código número y sus texto descriptivo, Metadatos de la información que se envió como el tipo de contenido, y en el body el cuerpo los datos estructurados en el JSON.

8. ¿Qué ventajas tiene utilizar Postman para probar una API antes de programarla?

Te permite definir el contrato de datos antes de tirar una sola línea de código en el backend. Puedes levantar servidores simulados para que el frontend empiece a trabajar de inmediato con respuestas ficticias y validar que las rutas y estructuras de datos tengan sentido antes de implementarlas.

9. ¿Qué es y para qué sirve Swagger UI?

Es una herramienta visual que genera una página web interactiva con la documentación de tu API. Sirve para ver todos los endpoints disponibles con sus parámetros y para probar peticiones directamente desde el navegador sin necesidad de herramientas externas.

10. ¿Qué función cumple Postman en el desarrollo y prueba de APIs?

Postman funciona como un cliente HTTP avanzado para depurar y automatizar pruebas. Te permite armar y mandar peticiones con headers y autenticaciones complejas, guardar colecciones de rutas organizadas por proyecto, cambiar entre entornos de desarrollo o producción rápidamente y correr scripts de prueba para verificar que el servidor responda correctamente.

1. Preparación del entorno

Instalar o utilizar Postman: <https://www.postman.com/downloads/>

API a explorar, Swagger Petstore 3: <https://petstore3.swagger.io/>

Importante: La actividad se realizará mediante solicitudes HTTP GET y análisis de las respuestas JSON. No se requiere API Key para esta práctica.

2. Exploración inicial

Ingresa a Swagger Petstore y localice las operaciones relacionadas con /pet

- Identifique un endpoint que permita consultar una mascota por ID.
- Indique el método HTTP utilizado.
- Indique qué parámetro necesita la solicitud.

POST	/pet	Add a new pet to the store.
GET	/pet/findByStatus	Finds Pets by status.
GET	/pet/findByTags	Finds Pets by tags.
GET	/pet/{petId}	Find pet by ID.

El método es /pet/{petId} y recibe como parámetro el ID del pet para encontrarlo



- Ejecute la solicitud con Try it out y registre el código HTTP recibido.

Name	Description
petId * required integer(int64) (path)	ID of pet to return 10

ExecuteClear

Responses

Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/10' \
  -H 'accept: application/json'
```

Request URL

```
https://petstore3.swagger.io/api/v3/pet/10
```

Server response

Code	Details
200	Response body<pre>{ "id": 10, "name": "Rufus", "photoUrls": ["example.com"], "tags": [] }</pre><button>Download</button> Response headers<pre>access-control-allow-headers: Content-Type,api_key,Authorization access-control-allow-methods: GET,POST,DELETE,PUT access-control-allow-origin: * access-control-expose-headers: Content-Disposition content-length: 62 content-type: application/json date: Fri, 02 Oct 2026 13:18:55 GMT server: Jetty (9.4.53.v20231009) x-firefox-spy: h2</pre>

- Explique con sus palabras qué información contiene la respuesta.

Se obtuvo un código 200, el cual indica que la solicitud fue exitosa, y la respuesta fue un JSON con la información del pet consultado, en este caso usé el ID 10, devolvió denombre "Rufus".

3. Crear un recurso con POST

Utilice POST /pet para crear una mascota. Construya un JSON propio; no copie exactamente el ejemplo de Swagger.

Curl

```
curl -X 'POST' \
  'https://petstore3.swagger.io/api/v3/pet' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
    "id": 11,
    "name": "Perry",
    "category": {
      "id": 11,
      "name": "Dogs"
    },
    "photoUrls": [
      "test.jpg"
    ],
    "tags": [
      {
        "id": 0,
        "name": "Bulldog"
      }
    ],
    "status": "available"
  }'
```

Request URL

```
https://petstore3.swagger.io/api/v3/pet
```



Curl

```
curl -X 'GET' \
  'https://petstore3.swagger.io/api/v3/pet/11' \
  -H 'accept: application/json'
```

Request URL

https://petstore3.swagger.io/api/v3/pet/11

Server response

Code	Details
200	Response body <pre>{ "id": 11, "category": { "id": 11, "name": "Dogs" }, "name": "Perry", "photoUrls": ["test.jpg"], "tags": [{ "id": 0, "name": "Bulldog" }], "status": "available" }</pre>

Download

4. Modificar el recurso

- Modifique la mascota creada utilizando PUT /pet. Cambie al menos dos atributos, por ejemplo nombre y estado.

Name	Description
petId required integer(int64) (path)	ID of pet that needs to be updated 10
name string (query)	Name of pet that needs to be updated Juan
status string (query)	Status of pet that needs to be updated Unavailable

Execute Clear

Responses

Curl

```
curl -X 'PUT' \
  'https://petstore3.swagger.io/api/v3/pet/10?name=Juan&status=Unavailable' \
  -H 'accept: application/json' \
  -d ''
```

Request URL

https://petstore3.swagger.io/api/v3/pet/10?name=Juan&status=Unavailable

Server response

Code	Details
200	Response body <pre>{ "id": 10, "name": "Juan", "photoUrls": ["example.com"], "tags": [], "status": "Unavailable" }</pre>

Download

- ¿Qué método HTTP utilizó y por qué? POST porque estoy subiendo datos publicandolos
- ¿Qué datos fueron enviados en el body? El JSON con los datos del pet a actualizar
- ¿Qué código HTTP devolvió la API? 200, Porque fue ejecutado exitosamente
- ¿Cómo comprobó que el cambio realmente ocurrió? Ejecuté un GET con el id del pet modificado.



5. Consultar utilizando parámetros

- Utilice GET /pet/findByStatus y consulte las mascotas con estado available.

```
Code  Details
200
Response body
{
  "id": 12345,
  "category": {
    "id": 1,
    "name": "dogs"
  },
  "name": "fluffy",
  "photoUrls": [],
  "tags": [
    {
      "id": 0,
      "name": "tag1"
    },
    {
      "id": 1,
      "name": "tag2"
    }
  ],
  "status": "available"
},
{
  "id": 123456,
  "category": {
    "id": 1,
    "name": "Dogs"
  },
  "name": "Dogs",
  "photoUrls": [],
  "tags": [],
  "status": "available"
}
```

- Identifique el parámetro utilizado. /pet/findByStatus?status=available
- Explique dónde se envía ese parámetro. Este se coloca al final del URL
- Observe la estructura de la respuesta. Un archivo JSON con todos los resultados estructuradamente
- Indique qué sucede si cambia el valor del parámetro por pending. Devuelve los pets con ese estado.

6. Reproducir las operaciones en Postman

- Ahora reproduzca en Postman las operaciones realizadas en Swagger. Cree una colección llamada 'MIA - Semana 13 - API'.
- La colección debe contener, como mínimo:
- GET – Consultar mascota por ID

```
GET https://petstore3.swagger.io/api/v3/pet/10
Query Params
Key Value Description
Key Value Description

Body Cookies Headers Test Results
200 OK 417 ms
JSON Preview Visualize
1 {
2   "id": 10,
3   "name": "Rufus",
4   "photoUrls": [
5     "example.com"
6   ],
7   "tags": []
8 }
```



- POST – Crear mascota

The screenshot shows a REST client interface with a POST request to `https://petstore3.swagger.io/api/v3/pet/`. The request body is a JSON object representing a pet. The response is a 200 OK status with a response body containing the created pet's details.

```
POST https://petstore3.swagger.io/api/v3/pet/

{
  "name": "dog",
  "photoUrls": [
    "test.jpg"
  ],
  "tags": [
    {
      "id": 0,
      "name": "Bulldog"
    }
  ],
  "status": "available"
}
```

Body Cookies Headers Test Results 200 OK 367 ms

```
{
  "id": 11,
  "category": {
    "id": 11,
    "name": "Dogs"
  },
  "name": "Perry",
  "photoUrls": [
    "test.jpg"
  ],
  "tags": [
    {
      "id": 0,
      "name": "Bulldog"
    }
  ],
  "status": "available"
}
```

- POST – Modificar mascota

The screenshot shows a REST client interface with a POST request to `https://petstore3.swagger.io/api/v3/pet/10?name=Juan&status=Unavailable`. The request body is a JSON object representing a pet. The response is a 200 OK status with a response body containing the updated pet's details.

```
POST https://petstore3.swagger.io/api/v3/pet/10?name=Juan&status=Unavailable

{
  "id": 10,
  "name": "Juan",
  "photoUrls": [
    "example.com"
  ],
  "tags": [],
  "status": "Unavailable"
}
```

Body Cookies Headers Test Results 200 OK

```
{
  "id": 10,
  "name": "Juan",
  "photoUrls": [
    "example.com"
  ],
  "tags": [],
  "status": "Unavailable"
}
```



- GET – Buscar por status

MIA - Semana 13 - API > **Buscar por status**

GET <https://petstore3.swagger.io/api/v3/pet/findByStatus?status=pending>

Docs Params Authorization Headers Body Scripts Tests Settings

Query Params

Key	Value	Description
status	pending	
Key	Value	Description

Body Cookies Headers Test Results

200 OK - 318 ms

JSON Preview Visualize

```
1 {
2   {
3     "id": 100,
4     "category": {
5       "id": 10000,
6       "name": "Perros"
7     },
8     "name": "Kobe",
9     "photoUrls": [
10      "String"
11    ],
12    "tags": [
13      {
14        "id": 100,
15        "name": "Doki"
16      }
17    ],
18    "status": "pending"
19  }
20 }
```

- DELETE – Eliminar mascota

DELETE <https://petstore3.swagger.io/api/v3/pet/10>

Docs Params Authorization Headers Body Scripts Tests Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers Test Results

200 OK - 508 ms

JSON Preview Visualize

```
1 Petdeleted
```



7. Prueba de errores

GET Consultar Mascota por ID

https://petstore3.swagger.io/api/v3/pet/11

GET Petstore OpenAPI (Example OpenAPI Specification) · https://petstore3.swagger.io/api/v3/openapi.json · Postman Atlas

Key	Value	Description
Key	Value	Description

Body Cookies Headers Test Results 404 Not Found · 546 ms

XML Preview Debug with AI

1 Pet not found

POST Crear Mascota

https://petstore3.swagger.io/api/v3/pet/

Docs Params Authorization Headers Body Scripts Tests Settings

none form-data x-www-form-urlencoded raw binary GraphQL JSON

```
1 {
2   "name": "Dogs"
3 }
4 {
5   "photoUrls": [
6     "test.jpg"
7   ],
8   "tags": [
9     {
10      "id": 0,
11      "name": "Bulldog"
12    }
13  ]
14 }
```

Body Cookies Headers Test Results 500 Internal Server Error

JSON Preview Debug with AI

```
1 {
2   "code": 500,
3   "message": "There was an error processing your request. It has been logged (ID: 29df8131e1524db2)"
4 }
```

MIA - Semana 13 - API · Eliminar Mascota

GET

https://petstore3.swagger.io/api/v3/pet/11

Docs Params Authorization Headers Body Scripts Tests Settings

Query Params

Key	Value	Description
Key	Value	Description

Body Cookies Headers Test Results 404 Not Found · 963 ms

XML Preview Debug with AI

1 Pet not found